

1. System Overview

UrbanDrop is an intelligent communal parking management system designed to help delivery drivers locate legal temporary parking locations in dense urban areas. The system identifies nearby parking spaces, manages parking availability, and generates routes to guide drivers to available locations.

The system architecture is built using object-oriented design principles to ensure modularity, maintainability, and scalability.

2. Class Design (Well-Designed Classes)

The UrbanDrop system contains the following main classes:

- User
- SupportRequest
- ParkingSpot
- Location
- Route
- UrbanDropSystem

Each class is designed with meaningful names, well-defined member variables, and clear member functions.

2.1 User Class

Member Variables

- `userId` : String
- `name`: String
- `role`: String
- `isAuthenticated`: boolean
- `currentLocation`: Location
- `bookmarkedSpots`: List<ParkingSpot>

Member Functions

- `bookmarkParkingSpot(spot : ParkingSpot)`
- `updateLocation(location : Location)`
- `requestRouteTo(destination : ParkingSpot) : Route`

The User class manages driver information and interactions with the system.

2.2 SupportRequest Class

Member Variables

- requestId : String
- user: User
- message: String
- status: String
- createdAt: DateTime

Member Functions

- updateStatus(newStatus : String)

This class represents user-generated support inquiries.

2.3 ParkingSpot Class

Member Variables

- spotId : String
- coordinates: Location
- availabilityStatus: boolean
- description: String
- lastUpdated: DateTime

Member Functions

- isAvailable() : boolean
- updateAvailability(status : boolean)
- getDetails() : String

ParkingSpot manages the availability and information of parking locations.

2.4 Location Class

Member Variables

- latitude: double
- longitude: double

Member Functions

- distanceTo(other : Location) : double
- getLatitude() : double

- getLongitude() : double

Location centralizes geographic coordinate calculations.

2.5 Route Class

Member Variables

- routeId : String
- pathCoordinates: List<Location>
- estimatedTime: double
- totalDistance: double

Member Functions

- calculateDistance() : double
- getSummary() : String

A route represents navigation information between locations.

2.6 UrbanDropSystem Class

Member Variables

- users: List<User>
- parkingSpots: List<ParkingSpot>
- supportRequests: List<SupportRequest>

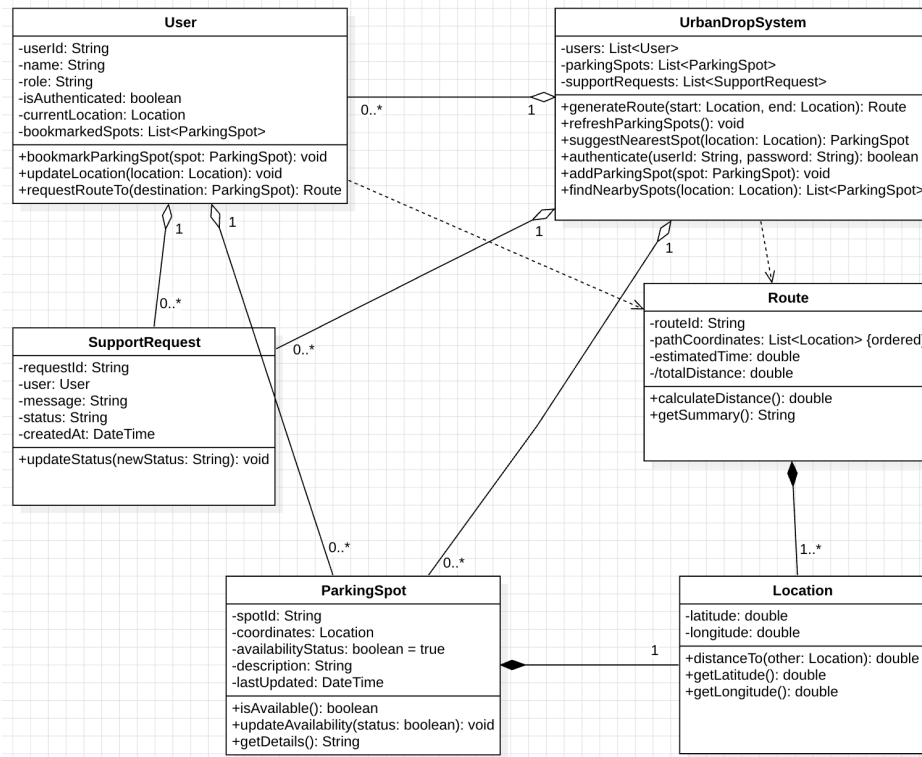
Member Functions

- generateRoute(start : Location, end : Location) : Route
- refreshParkingSpots(): void
- suggestNearestSpot(location : Location) : ParkingSpot
- authenticateUser(userId : String, password : String) : boolean
- addParkingSpot(spot : ParkingSpot)
- findNearbySpots(location : Location) : List<ParkingSpot>

UrbanDropSystem coordinates system operations and interactions between domain objects.

3. UML Class Diagram

The UML diagram below illustrates the relationships between the system classes.



The diagram includes:

- Class attributes and methods
- Associations between classes
- Cardinality (multiplicity) relationships
- Aggregation and composition relationships

For example:

A User can create multiple SupportRequests (1 → 0..)*

UrbanDropSystem manages many ParkingSpots

- **Route contains multiple Location objects (composition)**

3.1 Class Relationships

The UML diagram illustrates several types of relationships between classes.

Association

A User can create multiple SupportRequests. This relationship is shown with cardinality 1 → 0..*, meaning one user may generate many support requests.

Aggregation

The UrbanDropSystem aggregates Users, ParkingSpots, and SupportRequests. These objects exist independently but are managed by the UrbanDropSystem.

Example:

UrbanDropSystem o-- User

UrbanDropSystem o-- ParkingSpot

Composition

The Route class is composed of multiple Location objects representing path coordinates. A Location in this context exists only as part of a Route.

Example:

*Route *-- Location*

Dependency

The User class depends on UrbanDropSystem for services such as route generation and parking spot discovery. This dependency ensures that business logic remains centralized.

4. Cohesion (Single Responsibility)

Each class in the system performs a single well-defined responsibility.

Examples:

- **User** manages user interactions and state.
- **SupportRequest** manages customer support tickets.
- **ParkingSpot** manages parking availability.
- **Location** performs geographic calculations.
- **Route** manages navigation paths.
- **UrbanDropSystem** coordinates system-level operations.

This ensures high cohesion and improves maintainability.

5. Loose Coupling (Minimal Dependencies)

Classes interact through clearly defined interfaces rather than direct implementation dependencies.

Examples:

- The **User class does not compute routes directly**. It requests route generation from UrbanDropSystem.
- **ParkingSpot does not perform distance calculations**. These are handled by the Location class.
- **Route uses Location objects rather than accessing ParkingSpot internals**.

Loose coupling allows system components to evolve independently.

6. Hidden Implementation (Encapsulation)

Encapsulation hides internal implementation details from other classes.

Key design principles include:

- Member variables are private.
- External classes interact only through public methods.
- Geographic calculations are encapsulated within the Location class.
- Route distance calculations are performed internally by the Route class.

Encapsulation protects system data and prevents unintended modifications.

7. Encapsulation of Change

The system design isolates potential areas of change.

Authentication Changes

If the authentication mechanism changes, modifications are limited to the UrbanDropSystem class.

Navigation Algorithm Changes

Changes to routing algorithms are isolated within the Route class.

Parking Data Changes

If additional attributes such as pricing or time limits are added to parking spots, only the ParkingSpot class needs modification.

This design reduces system-wide impact when implementing future changes.